Hub-Aware Incremental Fraud Ring Detection on Streaming Transaction Graphs

Roshan Parveen
School of Computer Science and Engineering
Vellore Institute of Technology
Chennai, India
roshan.parveen2023@vitstudent.ac.in

Sanjay Kumar S
School of Computer Science and Engineering
Vellore Institute of Technology
Chennai, India
sanjaykumar.s2023@vitstudent.ac.in

***Abstract*—Fraud rings share devices, cards and network addresses and move money among themselves. Rule-based graph detectors cap the degree of shared attributes to suppress office gateways and carrier NAT, which hides any ring behind such a hub, and re-running whole-graph queries gets costly as the graph grows. We keep hub links with a degree-discounted weight, detect incrementally on the region each micro-batch touches, and score candidate rings with a calibrated, explained model. On a deterministic simulator with ring-level ground truth and five evasion strategies (5 scenarios, 10 paired test seeds), the pipeline reaches ring F1 0.73 against 0.21 for degree-capped Cypher rules; given the same scorer, the capped rules reach 0.56, so the scorer accounts for most of the gap. Keeping hub links raises the reachable recall on shared-attribute rings from 0.53 to 0.81, while the exact discount matters little. Incremental detection buys nothing over a 1 s timer-driven full pass on small graphs, but stays affordable as the graph grows, where a 5 s timer cannot keep up. Slow and identity-rotating rings evade every method we tested, and all positive results are synthetic.**

***Index Terms*—fraud rings, property graphs, streaming graph analytics, incremental detection, hub-aware weighting, explainability**

I. Introduction

Fraud rings are groups of accounts that act together: they share devices, payment cards or network addresses, and they move money among themselves in cycles or fan-in and fan-out patterns. Graphs make these relations explicit, which is why graph-based methods dominate fraud and anti-money laundering (AML) research [1], [2]. Two problems appear as soon as such methods run on a live stream of events.

The hub dilemma. Legitimate shared infrastructure, such as an office gateway, a carrier-grade network address translation (NAT) address or a store kiosk, links many unrelated accounts. Linking accounts by shared properties produces sparse links through strongly identifying entities and over-linkage through widely shared ones such as IP addresses [3]. The usual remedy in rule-based systems is a hard degree cap: attributes shared by more than c accounts are ignored. The cap removes the noise but also opens an evasion path, because a ring that shares a busy attribute becomes invisible to it, much as fraudsters elsewhere hide by connecting to benign entities [4].

Latency on a stream. Accounts, links and transfers arrive continuously. Re-running whole-graph queries on a timer delays detection, and the cost of each pass grows with the graph rather than with the change, a problem that incremental community detection addresses for clustering [5], [6].

We make four contributions.

- **C1, hub-aware weighting.** We replace the degree cap with a continuous weight: each shared attribute contributes $1/\ln(1+d)$, where d is its degree, with exponential time decay, and transfers inside the pair add weight. The discount itself is FRAUDAR's column weighting [7]; our contribution is its use for shared-attribute ring detection on a stream. Busy hubs count for little on their own, but a group that shares a hub and also trades among itself still accumulates weight (Section III). What we can show is that keeping hub links is what matters: a constant weight with its own threshold performs as well, and caps lose rings hidden behind hubs (Section V).
- **C2, incremental detection.** A micro-batch engine re-evaluates only the k-hop region touched by each batch, with fan-out limits that keep that region bounded when it reaches a hub. Against a timer-driven full pass it pays off only as the graph grows (Section V).
- **C3, calibrated, explained alerts.** Candidate rings are scored by a calibrated classifier, and each alert carries feature contributions, an evidence subgraph and counterfactuals. The scorer supplies most of the precision, for our candidates and for the baselines' (Section V).
- **C4, evaluation under evasion.** A deterministic simulator with ring-level ground truth and five evasion strategies, including an attacker that adapts to the detector's flags, lets us measure each component on identical event streams (a paired design) with confidence intervals.

II. Related Work

Graph fraud detection. Most recent graph fraud detection work trains graph neural networks (GNNs) to label nodes. It addresses heterophily, where fraudulent nodes connect mostly to benign ones [8], [9]; camouflage, by reweighting neighbours adversarially [4]; scarce or unreliable labels, with one-class and semi-supervised training [10]–[12]; and interpretability [13]. These methods output per-node scores and need labelled training graphs. GB-FRD [2] targets fraud rings in peer-to-peer payment graphs, as we do, and Yin et al. [3] replace hard shared-property links with behavioural similarity because of

over-linkage through shared entities. We keep shared-attribute links, which are cheap and explainable, and change how much each one counts.

Dense subgraphs and peeling. Finding unusually dense groups is a classical route to fraud and collusion detection: greedy peeling approximates the densest subgraph [14], spectral and greedy densest-subgraph algorithms scale to billion-edge graphs [15], and sequential peeling of a weighted bipartite graph exposes coordinated adversarial annotators [16]. FRAUDAR [7] makes peeling resistant to camouflage by weighting each edge by a decreasing function of the degree of the object it points to, recommending $1/\log(x+c)$ and arguing that $1/x$ shrinks too fast; Spade [17] maintains such peeling-based dense subgraphs incrementally under edge insertions on transaction graphs. Core decomposition, the peeling primitive we use to extract dense cores, has efficient parallel and GPU implementations [18], [19] and is related to densest-subgraph approximation [20]. Our degree discount is therefore not new: it is FRAUDAR's column weighting, and close to Adamic–Adar. What we study is applying it to shared-attribute ring detection on a stream, as a replacement for the degree cap of rule-based systems, with incremental detection, ring tracking and calibrated alerts around it.

Community detection on dynamic graphs. Louvain modularity optimisation [21] remains a standard way to find groups and is still being refined [22]; we use it as a baseline. Incremental community detection avoids repartitioning a changing graph by updating only the affected communities [5], [6]. Community structure, however, is not ring structure: a benign hub joins thousands of users into one community.

Similarity weighting. Local similarity indices such as Jaccard and Adamic–Adar [23], which discounts a shared neighbour by the logarithm of its degree, are standard in link prediction [24]. Our hub weight applies a similar discount to shared attributes, and we match flagged rings to true ones by Jaccard overlap.

AML on public data. Work on the Elliptic Bitcoin graph compares tabular ensembles with GNNs under temporal splits [25]–[28]; AMLworld provides synthetic laundering attempts with typology labels [29]. These studies classify transactions, so their scores are not comparable with our ring-level metrics (Section V-F).

Calibration and explanations. Post-hoc calibration by isotonic regression [30] and its comparison with Platt scaling [31] make classifier scores usable as probabilities; Azbergen et al. [32] compare such calibrators for AML risk scores, and learned calibration has been studied beyond classification [33]. Counterfactual explanations state what would have to change for a different decision; their robustness [34], consistency across model updates [35] and per-feature importance [36] are active topics. We attach simple, model-specific counterfactuals to every alert.

III. Method

Fig. 1 shows the system. A seeded simulator emits observable events (account joins and leaves, attribute links, transfers); ground truth goes only to a ledger the detector never sees.

A. Graph model

Users link to devices, cards and IP addresses (`USED_DEVICE`, `HAS_CARD`, `LOGGED_IN_FROM`) and send transfers to each other. The same contract is implemented over Neo4j (Cypher) and over an in-memory store with identical semantics; all experiments below use the in-memory store. In live runs, events reach the engine through an in-process queue or Redis Streams, whose consumer settings (batch size, blocking timeout) trade throughput against latency [37].

B. Hub-aware weighting (C1)

For two users u, v the edge weight is

$$W(u,v) = \sum_{a \in A(u) \cap A(v)} h(\deg a)\, \delta(t_a) + \min\Big(\beta \sum_{x \in T(u,v)} \delta(t_x),\, \beta_{\max}\Big), \tag{1}$$

where $h(d) = 1/\ln(1+d)$, $\delta(t) = 2^{-t/\tau}$ is exponential decay with half-life $\tau = 300\,\mathrm{s}$, t_a is the age of the older of the two links to attribute a, and $T(u,v)$ are the transfers between u and v in the window, with $\beta = 0.15$ and $\beta_{\max} = 0.6$. A device shared by three accounts weighs 0.72; an office address shared by fifty weighs 0.25. Candidate rings are the connected components of the graph of pairs with $W \geq \theta_e$ (and a non-zero attribute term), peeled to cores of edge density at least 0.75 and size 3–25. The edge threshold θ_e is selected on validation seeds (Section IV-B).

C. Incremental detection (C2)

Events are grouped into micro-batches of 50. For each batch the engine takes the users it touched, expands them by $k = 2$ hops through shared attributes and transfers, and builds a view of that region over a $900\,\mathrm{s}$ window, together with the *global* degree of every attribute in it, so $h(\deg a)$ is the same as in a full pass. The first hop may cross attributes with up to 250 users (so the co-members of a hub are re-examined); later hops cross only attributes with at most 20 users, which stops the region from spreading through hubs. The first-hop limit is itself a degree cap on the region: a pair whose only link is an attribute shared by more than 250 users never enters the same region. For the hub-aware weight this costs nothing, since one such attribute contributes at most $1/\ln 252 \approx 0.18$, below the edge threshold, so a full pass would not link the pair either; a weight that lets one busy attribute reach the threshold, such as a constant, does lose such pairs (Section V). Incremental detectors (hub-aware shared attribute, temporal cycle, smurfing) run on the view; whole-graph detectors (the static Cypher rules and community detection) run on a timer.

D. Ensemble, tracking and scoring (C3)

Candidates from different detectors are merged when their Jaccard overlap is at least 0.5, or when one contains 80% of the other and they differ in size by at most a factor of two. A tracker matches candidates to existing rings across

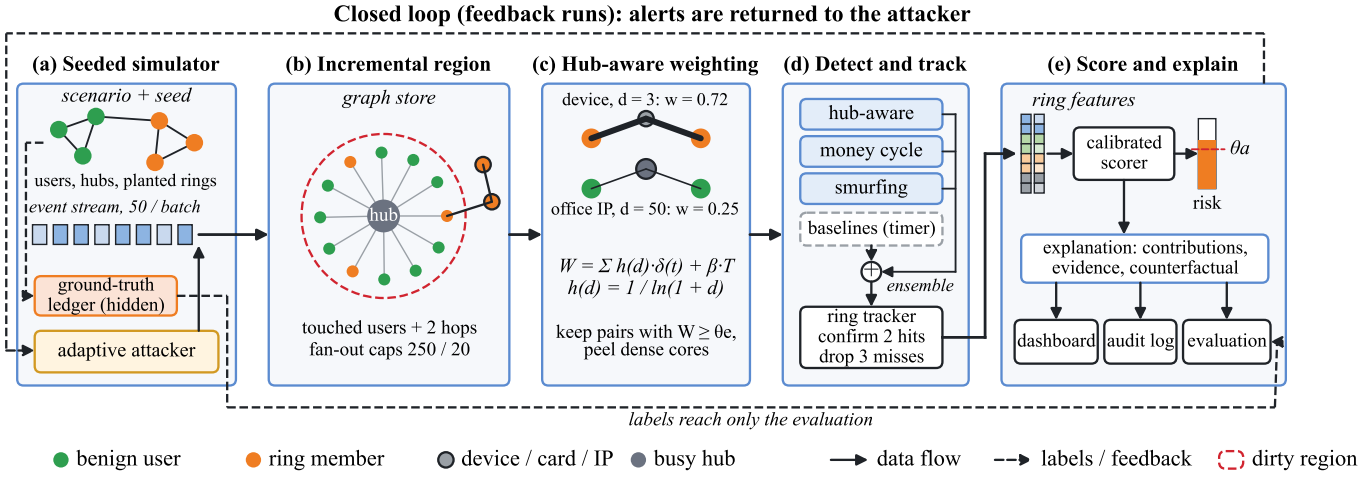


Fig. 1. Overview of the system. (a) A seeded simulator emits the observable event stream in micro-batches; labels go only to a ledger. (b) Each batch is written to the graph store and expanded to the bounded region it touched. (c) Shared attributes are weighted by their degree, so a busy hub counts for little and a rarely shared device for much. (d) Incremental detectors run on the region and a tracker confirms rings across batches. (e) Confirmed rings are scored, explained and alerted. Solid arrows: data flow; dashed arrows: labels (evaluation only) and, in closed-loop runs, alerts returned to the attacker.

batches; a ring is *confirmed* after two consecutive hits and dissolved after three misses (this hysteresis is what the ablation "w/o hysteresis" sets to one hit and one miss). Confirmed rings are scored by a logistic-regression model over 19 ring features (size, density, which attribute kinds are shared, the hub-adjusted weight, cycle presence and length, burstiness, account ages, internal transfers per member, amount statistics, growth, hub exposure and the number of agreeing detectors), calibrated with isotonic regression [30]. A ring becomes an alert when its calibrated risk reaches the alert threshold θ_a. Each alert carries the per-feature contributions to the score, the subgraph of shared attributes and transfers that formed it, and counterfactuals (for the leading features, the value at which the risk would fall below the threshold, and for hub-aware rings the degree the shared attribute would need before the rule stops firing). The model is trained on seeds disjoint from all evaluation seeds (Fig. 2): the detector runs on six scenarios with seeds 101–110, each confirmed or changed ring becomes a row of its 19 features, labelled from the hidden ledger, and within each run the first 70% of rows fit the regression and the last 30% the calibrator. On the held-out test seeds 201–205 it reaches ROC AUC 0.982 and PR AUC 0.849 over 1,961 ring units (a unit's score is its highest calibrated score); over the 4,516 correlated snapshots of those rings the figures are 0.982 and 0.908. Trained without one scenario and tested on it, the scorer loses at most 0.01 ROC AUC (`adversarial`) and at most 0.09 ring-level PR AUC (`drift`), but all scenarios come from the same generator.

IV. Experimental Setup

A. Simulator and scenarios

The simulator is a deterministic discrete-event generator: a scenario file and a seed fix the whole stream, which we verify by a digest of the event sequence recorded with every result.

TABLE I
Scenarios (from `configs/scenarios/*.yaml`).

Scenario	Users	Dur. (s)	Ev./s	Rings and evasion
baseline	30	60	10	4 planted, none
high_noise	150	60	50	4 planted + random, decoys
hub_heavy	100	60	25	4 planted, hub hiding
adversarial	120	90	30	5 planted + random, all five
drift	200	90	40	2 planted + random, hub hiding
demo_story	120	90	20	3 planted, hub hiding
scale_sweep	10^3+	120	100	latency benchmark (RQ2)

Benign users join and leave, link to devices, cards and IPs drawn from heavy-tailed popularity, and transact; benign hubs (offices, carrier NAT, kiosks, family devices) are generated with configured sizes. Ring types are shared device, card or IP, mixed attribute, money cycle, layered chain and smurfing fan-in/fan-out. Evasion strategies are hub hiding (the ring shares a busy benign hub), low-and-slow activity, identity rotation, decoy edges to benign users, and an adaptive attacker that shifts its strategy mix away from strategies that were flagged. Fig. 3 shows the generator and its two outputs, and Table I lists the scenarios.

B. Protocol

Seeds are split three ways: scorer training (101–110, test 201–205), threshold selection (validation, 301–303) and evaluation (test, 1001 and up). On the validation seeds we evaluated a grid of alert thresholds $\theta_a \in \{0.2, 0.3, 0.5, 0.7, 0.8, 0.9\}$ and edge thresholds $\theta_e \in \{0.2, 0.3, 0.4, 0.5, 0.6\}$ on the five accuracy scenarios; the grid was widened whenever the best value sat on its edge. The selected values are $\theta_a = 0.8$ and $\theta_e = 0.3$ (mean validation F1 0.72). We report the proposed method both with these values and with the a-priori values $\theta_a = \theta_e = 0.5$.

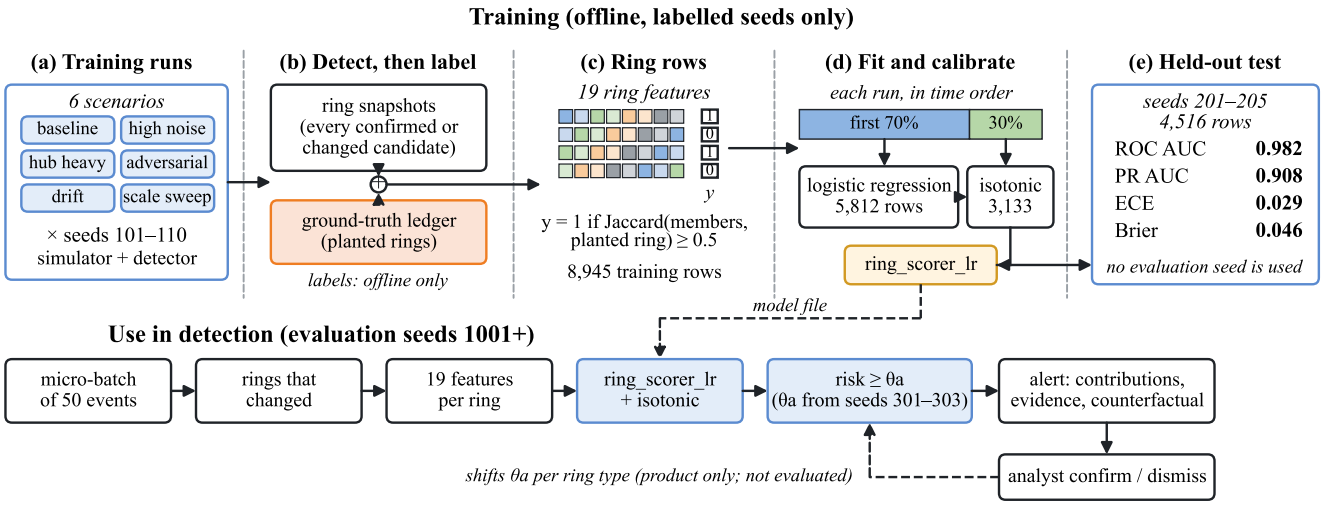


Fig. 2. Training and use of the ring scorer. (a)–(b) The simulator and detector run on training seeds, and ring snapshots are labelled from the ground-truth ledger, which the detector never reads. (c) Each snapshot is a row of 19 ring features. (d) Within each run, rows in time order are split 70/30 between logistic regression and isotonic calibration. (e) Held-out test seeds (values from `models/ring_scorer_lr.json`). Below: in detection, each changed ring is scored every batch, and an alert carries its explanation; analyst feedback adjusts the threshold in the product; that loop is not part of any experiment in this paper.

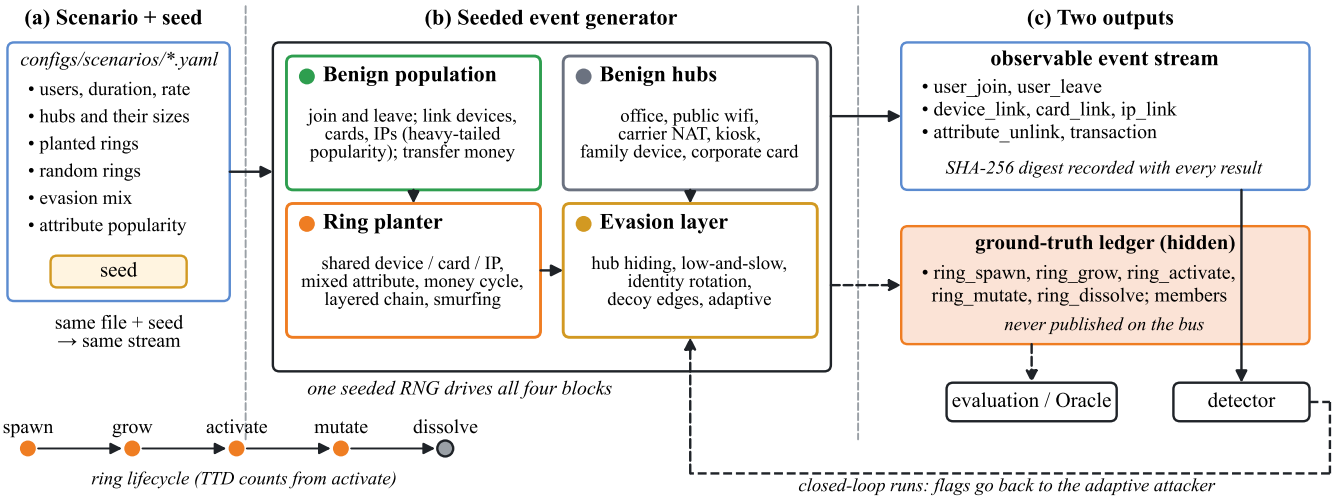


Fig. 3. Simulator design. (a) A scenario file and a seed fix the whole run. (b) One seeded generator combines a benign population, benign hubs, planted rings and an evasion layer. (c) Observable events go to the detector, with a digest of the sequence recorded with every result; ring lifecycle events go only to the ground-truth ledger used for evaluation and for the demo's Oracle view, which overlays that ground truth on the dashboard and is not shown to the detector. In closed-loop runs, flags are returned to the adaptive attacker.

Every method in an experiment processes the *identical* event stream for a given scenario and seed (the attacker does not receive feedback in these runs), so differences between methods are paired. We report the mean and 95% t-interval over runs, and test differences with the Wilcoxon signed-rank test on paired runs, with Cohen's d_z as effect size.

C. Metrics

A flagged ring *matches* a true ring when their member sets overlap with Jaccard at least 0.5. Ring precision is the fraction of flagged rings that match a true ring; ring recall is the fraction of true rings matched at some point while active. A run in which nothing is flagged has undefined precision; we count its F1 as 0. Time to detect (TTD) is the time from a ring's activation to its first matching flag. Detection latency is the wall-clock time per micro-batch for the dirty-region expansion, the view and the incremental detectors.

D. Baselines

- **Static Cypher, cap 100 / cap 10.** The original rules: shared device (Q1), shared card (Q2), 3-cycles of transfers (Q3), and shared IP with a degree cap (Q4a/Q4b), run on the whole graph every 5 s of simulated time.
- **Louvain.** Louvain communities [21] of the user graph projected through shared attributes and transfers, kept when dense enough.
- **Connected components.** Weakly connected components of the same projection, no density filter.

All baselines use the same two-hit confirmation as the proposed method.

E. Environment

All runs used one laptop (Intel Core i3-1115G4, 2 cores / 4 threads, 7.7 GB RAM, Windows 11), Python 3.11.9, NumPy 2.2.6, scikit-learn 1.7.2, NetworkX 3.5, with experiments run one at a time.

V. Results

A. RQ1: accuracy against baselines

Table II pools 5 scenarios × 10 test seeds for every method, all on identical streams. Without a scorer, the static rules reach ring F1 0.21 with a degree cap of 10 and 0.19 with a cap of 100, Louvain 0.29 and connected components 0.00. These rows differ from the proposed method in two ways at once: how candidates are generated and whether they are scored. With the same calibrated scorer and an alert threshold selected per method on the validation seeds, the static rules reach 0.56 (cap 10) and 0.52 (cap 100) and Louvain 0.41, against 0.73 ($\pm$0.05) for the proposed method; every difference to the proposed method remains significant after Holm correction ($p < 0.001$). Within the scored group the differences isolate the candidate generator, the hub-aware rule and incremental detection; between a generator's unscored and scored rows they isolate the scorer. For the rules with cap 10, the scorer raises F1 from 0.21 to 0.56, through precision (0.14 to 0.64) at some cost in recall (0.61 to 0.53); the candidate generator raises it further to 0.73, mostly through recall (0.53 to 0.74). The scorer was trained on candidates of the proposed pipeline, so on rule and Louvain candidates it works under some distribution shift. The unscored static rules reach recall 0.61 but precision only 0.14: the shared-attribute rules and the 3-cycle rule fire on many benign groups, and two-hit confirmation does not remove them. Connected components never produce a group within the ring size limits, because benign hubs join almost all users into one component; Louvain splits that component, but its communities rarely coincide with a ring. The thresholds selected on validation seeds improve on the a-priori ones (F1 0.73 vs. 0.68, $p = 0.03$).

Per scenario (Table III), the gap to the static rules is largest on `drift` and `high_noise`, where many benign groups satisfy the rules and their precision collapses, and smallest on the clean `baseline` scenario and on `adversarial`, which is the hardest scenario for every method.

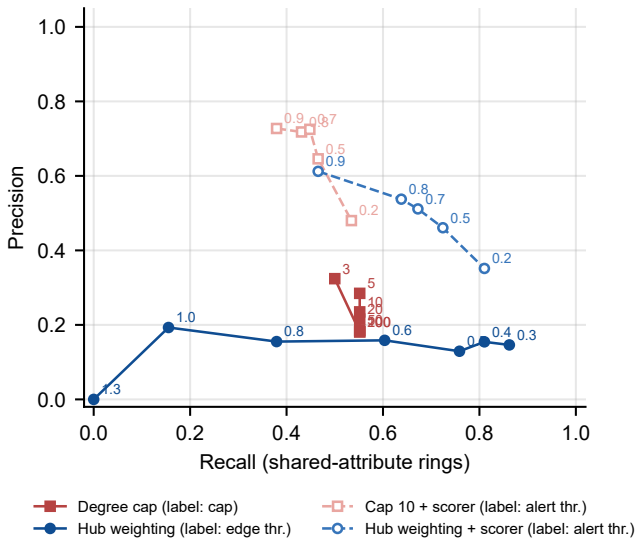


Fig. 4. Precision and recall on shared-attribute rings (micro-averaged over 4 scenarios × 5 test seeds). Solid: detector alone, swept over the degree cap or the edge threshold θ_e. Dashed: with the calibrated scorer, swept over the alert threshold θ_a (cap 10; $\theta_e = 0.3$). Best-F1 difference with the scorer (hub minus cap): 0.03, 95% bootstrap interval -0.03 to 0.10 (`results/e3b_frontier_ci`).

Degree cap vs. hub weighting. Fig. 4 isolates the mechanism this paper replaces. Only shared-attribute and mixed rings count as ground truth here (flags that match only other ring types are ignored), the static rules run without the cycle rule, and the hub-aware detector runs alone. Without the scorer, no degree cap reaches a recall above 0.55: rings that share a busy attribute are invisible to every cap. Hub weighting reaches recall 0.86, but on its own at precision 0.15. With the calibrated scorer on top, hub weighting reaches recall 0.81 (precision 0.35) and its best F1 is 0.58 (alert threshold 0.8); the cap with the scorer stays below recall 0.53 and its best F1 is 0.55. The difference in best F1, 0.03, has a 95% bootstrap interval of -0.03 to 0.10 over the 20 runs (the best threshold re-selected in every resample), so at the best operating point the two mechanisms are not distinguishable on F1. At low recall the cap with the scorer is the more precise of the two (0.72 at recall 0.45). Hub weighting therefore does not dominate the cap everywhere; it extends the frontier to recall levels the cap cannot reach, and the scorer is what makes those levels usable.

B. RQ2: latency and scale

Table IV and Fig. 5 grow the user population of the `scale_sweep` scenario at a fixed event rate (100 events/s) and compare incremental detection with recomputing the same detectors on the whole graph for every batch. Full recomputation grows at least linearly with the graph (faster between 50,000 and 100,000 users, where the 7.7 GB test machine was short of memory; we did not separate the two effects). The incremental region stays between a few hundred and two thousand users, because the fan-out limits stop the expansion at hubs; it even shrinks as the graph grows, since this scenario

TABLE II

RING-LEVEL DETECTION ON 5 SCENARIOS × 10 TEST SEEDS, POOLED (MEAN ± 95% CI OVER RUNS; A RUN WITH NO FLAGS HAS UNDEFINED PRECISION AND COUNTS AS F1 = 0). TTD: MEDIAN TIME FROM RING ACTIVATION TO FIRST FLAG. LATENCY: MEDIAN PER-BATCH DETECTION TIME ON THE IN-MEMORY STORE; †FULL-GRAPH METHODS RUN ON A 5 S TIMER, NOT PER BATCH, SO THEIR ENTRY IS THE MEDIAN TIME OF ONE FULL-GRAPH PASS. p: PAIRED WILCOXON SIGNED-RANK TEST OF F1 AGAINST THE PROPOSED METHOD ON IDENTICAL STREAMS. SCORED BASELINES USE THE SHIPPED SCORER (`MODELS/RING_SCORER_LR.JSON`), WITH THE ALERT THRESHOLD SELECTED PER METHOD ON THE VALIDATION SEEDS; THE PROPOSED METHOD INCLUDES THE SAME SCORER, SO ONLY ROWS WITHIN A GROUP ISOLATE THE CANDIDATE GENERATOR. p VALUES ARE HOLM-CORRECTED.

Method	Precision	Recall	F1	TTD (s)	Latency (ms)	p
Without a scorer						
Static Cypher (cap 100)	0.12 ± 0.03	0.61 ± 0.08	0.19 ± 0.05	9.5	6.0†	<0.001
Static Cypher (cap 10)	0.14 ± 0.04	0.61 ± 0.08	0.21 ± 0.05	9.5	5.9†	<0.001
Louvain communities	0.39 ± 0.09	0.33 ± 0.08	0.29 ± 0.06	18.8	38.4†	<0.001
Connected components	–	0.00 ± 0.00	0.00 ± 0.00	–	11.1†	<0.001
With the calibrated scorer						
Static Cypher (cap 100) + scorer	0.63 ± 0.08	0.48 ± 0.09	0.52 ± 0.08	8.6	4.5†	<0.001
Static Cypher (cap 10) + scorer	0.64 ± 0.07	0.53 ± 0.09	0.56 ± 0.07	9.3	4.6†	<0.001
Louvain communities + scorer	0.81 ± 0.12	0.32 ± 0.09	0.41 ± 0.09	19.0	27.9†	<0.001
Hub-aware incremental (proposed)	0.76 ± 0.06	0.74 ± 0.06	**0.73 ± 0.05**	9.3	26.6	–
Proposed, a-priori thresholds	0.73 ± 0.06	0.69 ± 0.06	0.68 ± 0.05	9.0	22.7	0.034

TABLE III

RING F1 PER SCENARIO (MEAN ± 95% CI OVER 10 TEST SEEDS).

Method	baseline	high_noise	hub_heavy	adversarial	drift
Static Cypher (cap 100)	0.48 ± 0.07	0.06 ± 0.02	0.24 ± 0.04	0.14 ± 0.04	0.05 ± 0.02
Static Cypher (cap 10)	0.54 ± 0.08	0.07 ± 0.02	0.27 ± 0.05	0.14 ± 0.04	0.05 ± 0.02
Louvain communities	0.44 ± 0.09	0.26 ± 0.19	0.24 ± 0.09	0.14 ± 0.09	0.38 ± 0.16
Connected components	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00
Hub-aware incremental (proposed)	0.99 ± 0.03	0.62 ± 0.08	0.79 ± 0.08	0.59 ± 0.14	0.67 ± 0.04
Proposed, a-priori thresholds	0.93 ± 0.07	0.66 ± 0.05	0.66 ± 0.12	0.53 ± 0.11	0.62 ± 0.09

TABLE IV

PER-BATCH DETECTION LATENCY (MEDIAN, MS; MEAN ± 95% CI OVER 3 SEEDS) AND USERS EXAMINED PER BATCH, INCREMENTAL VS FULL RECOMPUTATION, IN-MEMORY STORE.

	Incremental		Full		
Users	Region	ms	Region	ms	Speed-up
1,000	976	211.0 ± 142.9	1015	159.7 ± 64.4	0.8×
5,000	1878	177.8 ± 46.7	5016	589.8 ± 67.3	3.3×
20,000	1324	54.6 ± 37.1	20010	2683.7 ± 1062.0	49.2×
50,000	1110	38.5 ± 38.2	50011	6181.6 ± 2576.4	160.4×
100,000	827	36.1 ± 44.2	100013	22500.5 ± 10278.5	623.0×

scales the number of devices, cards and IPs with the users (so an ordinary attribute is shared by fewer users) while hub sizes stay fixed. Its latency therefore stays flat: at 100,000 users a batch takes 36.1 ms incrementally against 22,501 ms in full. This comparison is a worst-case bound, not a deployment comparison: nobody would recompute the whole graph on every 50-event batch, the incremental figure (3 seeds) varies more than its mean, and the 50,000 and 100,000-user full figures are confounded by memory pressure. At 1,000 users the region already covers almost the whole graph, and the incremental mode is slower than full recomputation because of the cost of expanding the region. On the accuracy scenarios, incremental and full recomputation flag exactly the same rings in every run (ablation *no incremental* in Table V), so restricting detection to the dirty region loses nothing there.

Against a timer. The realistic alternative is to run the same detectors on the whole graph on a timer. On the accuracy scenarios (30 to 200 users, 25 paired runs) a 1 s timer matches the incremental engine in F1 (0.69 vs. 0.71, $p = 0.624$) and time to detect (8.8 s vs. 9.5 s) at lower detection cost (22 vs. 37 ms per simulated second); a 5 s timer costs less still (6) but detects later (13.8 s), and a 30 s timer misses almost every ring, which ends before two passes confirm it (F1 0.01). On small graphs, incremental detection therefore buys nothing. The picture reverses with size. In `scale_sweep` with hubs that grow with the population (15% and 8% of users, other hubs up to 10%; 3 seeds), the cost per simulated second is 323, 464 and 173 ms for incremental detection at 1,000, 5,000 and 20,000 users, against 27, 371 and 4,170 ms for a 5 s timer (44,907 ms for recomputing every batch at 20,000). Above a few thousand users the timer needs more than one second of computation per simulated second, so it cannot keep up, while

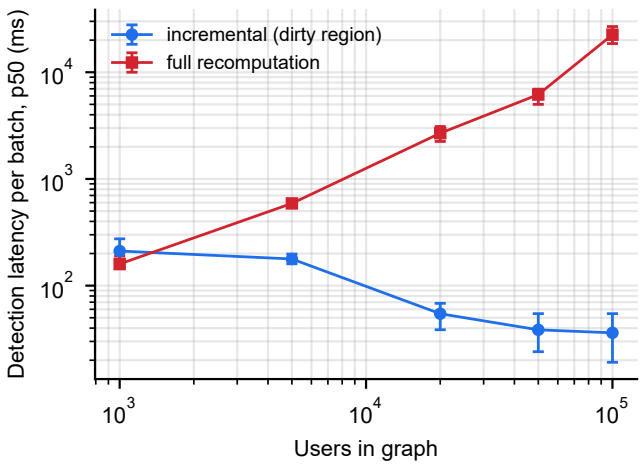


Fig. 5. Median per-batch detection latency as the graph grows (mean over 3 seeds; bars: per-seed minimum and maximum), incremental (dirty region) vs. full recomputation of the same detectors, in-memory store.

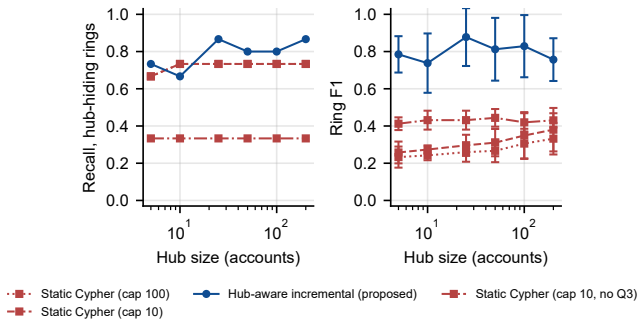


Fig. 6. A shared-IP ring hiding behind an office hub of growing size (`hub_heavy`, 5 test seeds per size). Left: recall of the hub-hiding rings; right: ring F1 (mean, 95% CI).

the incremental region stays near 912 users even though the hubs grow. The static rules run on the same 5 s timer, which is why their time to detect is similar to ours on the accuracy scenarios (9.3 s vs. 9.5 s).

C. RQ3: robustness to evasion

Hub size. In Fig. 6 the office hub grows from 5 to 200 accounts. Recall of the hub-hiding rings does not fall with hub size for any method (proposed 71/90, static cap 10 65/90 over all sizes). The static rules do not find these rings through the shared hub: with the 3-cycle rule switched off, cap 10 finds 30/90, because the rings also move money among themselves in short cycles. What separates the methods is precision, and so F1 (0.80 vs. 0.31).

Hubs above the fan-out limit. Fig. 6 stops at 200 accounts, below the 250-user first-hop limit. We therefore resized the office hub to 500, 2,000 and 10,000 accounts (5 seeds each) and compared the shipped engine with full recomputation and with a "bound" rule that does cross an over-limit attribute, towards those co-members whose attribute weight with the touched user could reach the edge threshold. At every size the bound rule flagged the same rings as the 250-user limit, with identical precision, recall and region size (15/15 runs): it never added a user. Full recomputation does not give identical runs, since it sees groups outside the region, but it found no more hub-hiding rings (11/15 vs. 11/15 at 500, 9/15 vs. 10/15 at 2,000) and its F1 was not higher (0.79 vs. 0.79, 0.80 vs. 0.83), while the incremental region stayed at a median of 14 users against up to 2,076. At 10,000 accounts one full-recomputation run did not finish within 3.5 h on the test laptop, so there is no full-recomputation reference at that size. The test does exercise the limit: on the same streams a constant weight (edge threshold 1.0, as selected in RQ5) found fewer hub-hiding rings with the limit than with full recomputation at 500 accounts (9/15 vs. 12/15; F1 0.74 vs. 0.85, lower in 3 of 5 runs; with 5 seeds not significant), and one full run of it took about 32 minutes at 2,000 accounts. So, for hubs up to 10,000 accounts, the limit did not hide hub-sharing rings for the hub-aware weight; for a constant weight it can. The static rules stay well behind (F1 0.46, 0.60, 0.63).

Evasion strategies. Fig. 7 breaks recall down by the strategy each planted or random ring uses. The proposed method's advantage in recall comes mostly from rings without evasion (91/110 vs. 67/110 for cap 10) and rings with decoy edges (22/27 vs. 13/27). On hub-hiding rings the two are close (35/52 vs. 33/52), and identity rotation (4/14) and low-and-slow activity (3/13) defeat every method we tested, presumably because rotation removes the shared attributes the detectors rely on and slow rings accumulate little evidence inside one window.

Adaptive attacker. With feedback switched on, the attacker is told which of its rings were alerted and shifts its strategy mix. Over 10 test seeds it switched strategy 22 times and launched 22 adaptive rings, of which 18 were detected; without feedback, 9 of 11. Adaptation did not lower the detection rate in our simulator, although its strategies are limited to the four we implemented.

D. RQ4: sensitivity

Raising the event rate from 10 to 100 events/s lowers the proposed method's F1 from 0.61 to 0.43 and raises its per-batch latency from 10.1 to 74.5 ms, since each batch touches more of the graph (Fig. 8). Larger planted rings are harder (F1 0.98 at size 3, 0.76 at size 8); a likely cause, which we did not isolate, is the 0.75 edge density the peeling requires. The view window between 60 and 900 s changes F1 little (0.64 vs. 0.67).

E. RQ5: ablations

Table V removes one component at a time (20 paired runs). The calibrated scorer contributes most: without it F1 falls from 0.72 to 0.39 because precision drops to 0.28. Without the hub-aware detector, F1 falls to 0.36 and to 0.00 on `hub_heavy`, but precision rises to 0.98 at recall 0.25: most false candidates come from the hub-aware detector, and the scorer removes most of them. A separate alert threshold per

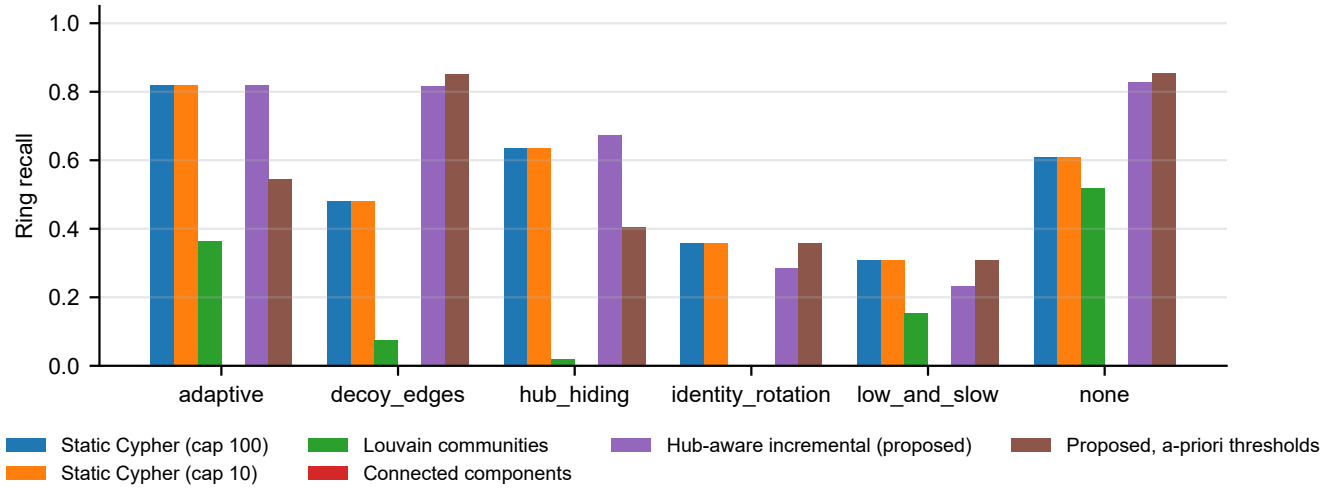


Fig. 7. Ring recall by evasion strategy, pooled over all runs of Table eftab:baselines.

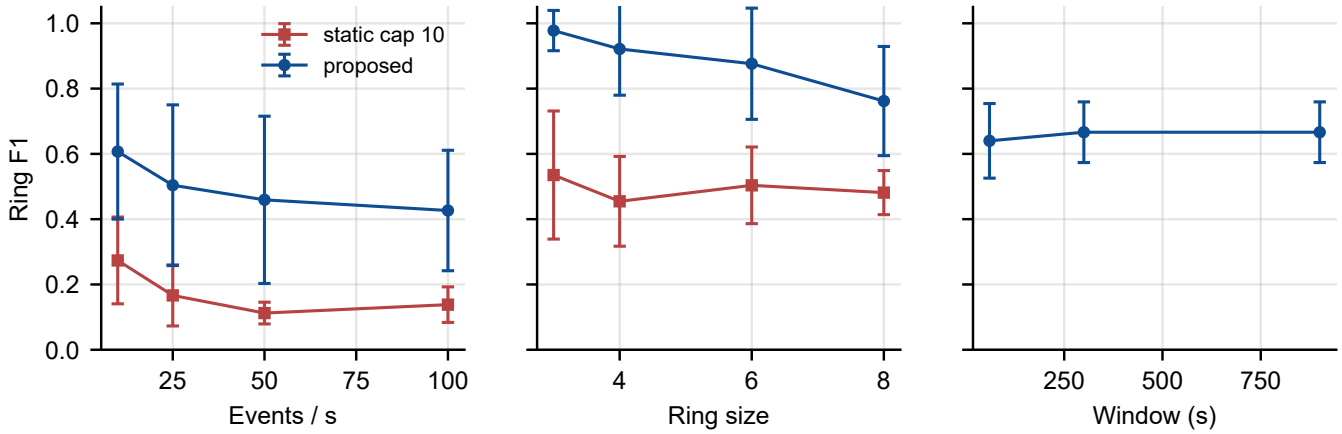


Fig. 8. Ring F1 of the proposed method and of the static rules (cap 10) when the event rate (`adversarial`), the planted ring size (`baseline`) and the view window (`drift`, proposed only) change; 5 test seeds per point.

detector is a cheap refinement this suggests; we have not tested it. Removing the cycle or the smurfing detector costs 0.64 and 0.65. Replacing the hub weight by a constant at the same edge threshold collapses `hub_heavy` (0.81 to 0.42) but slightly helps `high_noise` (0.54 to 0.68), so over all four scenarios the difference is not significant ($p = 0.161$). That ablation keeps the edge threshold tuned for $1/\ln(1+d)$, which is unfair to the constant; the next paragraph gives each weight its own threshold.

Weight functions. We compared five weights $h(d)$, each with its edge threshold selected on the validation seeds and the alert threshold fixed at 0.8: the shipped $1/\ln(1+d)$ ($\theta_e = 0.3$), $1/d$ (0.1), $1/\sqrt{d}$ (0.2), a soft cap $1/(1+e^{(d-10)/2})$ (1) and a constant (1); for $1/d$, the soft cap and the constant the selected value lies on the edge of the grid. On `hub_heavy` with 30 paired seeds, the shipped weight reaches F1 0.79 and recall 0.87 of the rings hidden behind hubs; the constant reaches 0.81 and 0.83 ($p = 0.249$), $1/\sqrt{d}$ 0.79, while $1/d$ (0.56) and the soft cap (0.60, hidden-ring recall 0.27 at precision 0.99) are worse (p <0.001). Over the five scenarios the picture is the same: 0.73 for the shipped weight, 0.76 for the constant and 0.74 for $1/\sqrt{d}$, none significantly different even before correction, and 0.62 for $1/d$ and 0.68 for the soft cap. Discounting busy attributes too strongly, or switching them off, loses rings that share a hub; within the range that keeps hub links, the shape of the discount makes no measurable difference once the edge threshold is tuned for it. Hysteresis and time decay make no significant difference here; without decay F1 is even slightly higher (0.75). The accuracy scenarios last 60–90 s, short against the 300 s half-life, so decay has little to act on here; its value on long streams is untested.

F. Public data

None of the public datasets we could obtain has device, card or IP links, so on all of them only the transaction detectors

TABLE V

ABLATIONS ON ADVERSARIAL, HUB_HEAVY, HIGH_NOISE, BASELINE ($\times$ 5 TEST SEEDS). ΔF1: PAIRED MEAN DIFFERENCE TO THE FULL SYSTEM; d_z: PAIRED EFFECT SIZE OF THE FULL SYSTEM OVER THE VARIANT; p: WILCOXON SIGNED-RANK, UNCORRECTED AND HOLM-CORRECTED OVER THE 7 TESTED COMPARISONS.

Variant	Precision	Recall	F1	ΔF1	d_z	p	p_{Holm}	Latency (ms)
Full system	0.75 $\pm$ 0.12	0.73 $\pm$ 0.12	0.72 $\pm$ 0.11	–	–	–	–	19.2
w/o hub weighting	0.65 $\pm$ 0.15	0.58 $\pm$ 0.14	0.60 $\pm$ 0.14	-0.12	0.42	0.161	0.462	24.0
w/o time decay	0.80 $\pm$ 0.11	0.72 $\pm$ 0.10	0.75 $\pm$ 0.09	+0.02	-0.18	0.277	0.462	17.6
w/o incremental scheduling	0.75 $\pm$ 0.12	0.73 $\pm$ 0.12	0.72 $\pm$ 0.11	0.00	–	–	–	18.4
w/o risk model	0.28 $\pm$ 0.09	0.82 $\pm$ 0.09	0.39 $\pm$ 0.10	-0.33	2.05	$<$0.001	$<$0.001	15.1
w/o hysteresis	0.72 $\pm$ 0.12	0.74 $\pm$ 0.10	0.70 $\pm$ 0.10	-0.02	0.37	0.154	0.462	22.7
w/o hub-aware detector	0.98 $\pm$ 0.05	0.25 $\pm$ 0.09	0.36 $\pm$ 0.12	-0.36	1.02	0.001	0.006	5.3
w/o cycle detector	0.75 $\pm$ 0.14	0.60 $\pm$ 0.11	0.64 $\pm$ 0.11	-0.09	1.02	0.002	0.009	15.4
w/o smurfing detector	0.72 $\pm$ 0.14	0.64 $\pm$ 0.15	0.65 $\pm$ 0.14	-0.07	0.93	0.003	0.013	19.9

(temporal cycle, smurfing) run, and the results below say nothing about hub-aware weighting (Table VI; sources and hashes in `docs/datasets.md`).

AMLworld [29] (HI-Small) is the only one with group labels: 370 laundering attempts of eight typologies. We replayed the first four days (2,284,182 transactions, 399,735 accounts) with a 1 h window and used the 55 attempts that lie entirely in that slice as ring ground truth. The detectors confirmed 303 groups; attempt recall is 0.00 and account-level precision and recall are 0.03 and 0.01. In other words, the transaction rules matched none of the attempts, and the groups they did confirm were almost all benign. Attempts span a median of 75 h, far longer than the window. A one-day window was too slow on our test machine (throughput fell steadily as the window filled), so attempts whose transfers are hours apart cannot be matched; and AMLworld's typologies (fan-in, scatter-gather, bipartite, stacks) are mostly not the relayed cycles and structuring our two transaction rules look for. The result is a negative one for short-window transaction rules on this data, not a comparison with published AMLworld models, which classify individual transactions.

Elliptic, as used in [25], [26], links each Bitcoin transaction to the later transactions spending its outputs, so its graph has no cycles, and amounts are not published. Neither transaction rule can fire: no account was flagged (0 of 203,769). We report it as not applicable to group detection rather than as a score.

AMLSim samples (IBM, 20,000 accounts, 200 planted cycles and/or fan-ins) were not flagged either (0, 0 and 0 accounts). The planted cycles are not time-ordered: none of the 21 we could reconstruct from equal hop amounts has its hops in time order (one cycle's five hops fall at steps 28, 113, 84, 59 and 78), whereas our cycle detector, by design, looks for money relayed around a loop.

VI. DEPLOYMENT, CALIBRATION AND EXPLANATION STUDIES

The experiments in this section were added after the main evaluation. Each has a protocol committed to the repository before its first result (`docs/research/`), and every amendment is recorded there with its reason.

A. RQ6: latency on the deployed stack

The latencies of Section V come from the in-memory store. We therefore replayed the paced scenarios of RQ1 and the users sweep through the deployed path: Redis Streams into the detection engine, with the graph in Neo4j 5.26. All 52 runs produced the same sequence of ring updates and the same flagged rings as the in-memory engine (52 of 52), so the store and the bus change timing, never results. Paced at simulated real time, a flag followed the event that completed its batch by a median of 176–438 ms across the five scenarios. Each detection reads its region through Cypher round trips and took 3 to 19 times as long as on the in-memory store for the same batches. When events arrive faster than the store can take them, Neo4j sets the ceiling: 35–52 events per second on the test laptop, with events queueing in Redis and flags arriving 9–19 s after their event. Varying the consumer's read size and blocking timeout, after HYADES [37], changed nothing measurable at these rates. Neo4j saturates before the consumer does.

B. RQ7: ring-level calibration and early warning

The scorer's isotonic calibration was fitted on snapshot rows, but analysts read scores of ring-level alerts. On 1383 ring-level units from the test seeds its expected calibration error is 0.10, and rings scored in the 0.8–0.9 and 0.9–1.0 bins were true rings 29% and 74% of the time. A calibrator refitted on ring-level units of separate seeds, in the manner compared by Azbergen et al. [32] and Ding et al. [33], lowers the Brier score from 0.085 to 0.044. Ranking holds up better than calibration: precision at the top 5% of units is 0.79. Early warning, measured without look-ahead, is rare. Of 227 true rings with a forming phase, 30 (13%) were flagged before activation, a median of 2.2 s early, and 84% of first early warnings pointed at a real ring.

C. RQ8: explanation quality

We measured the explanations the product shows on the 262 alerts of the test streams. The unit is an alert (a ring at its peak calibrated score of at least 0.8); a model counterfactual is a proposed change of one actionable feature, and an alert can have none, one or several. 81 of the 262 alerts have at least one, 103 in all. Writing the unit test for counterfactual

TABLE VI

PUBLIC DATA, TRANSACTION DETECTORS ONLY (NO DEVICE/CARD/IP LINKS EXIST). ACCOUNT PRECISION/RECALL: ACCOUNTS IN CONFIRMED GROUPS VS. ACCOUNTS IN LAUNDERING TRANSACTIONS. ATTEMPT RECALL: SHARE OF LAUNDERING ATTEMPTS THAT LIE ENTIRELY IN THE SLICE AND ARE AT LEAST HALF COVERED BY ONE FLAGGED GROUP.

Dataset	Slice	Rows	Attempts	Acc. precision	Acc. recall	Attempt recall
amlsim_20K_cycle200	all	117,805	0	–	0.00	–
amlsim_20K_fanin200	all	118,250	0	–	0.00	–
amlsim_20K_fanin200cycle200	all	120,558	0	–	0.00	–
elliptic	all	234,355	0	–	0.00	–
amlworld	2022-09-01 to 09-05 (excl.)	2,284,182	55	0.03	0.01	0.00
attempt recall by typology: bipartite 0.00, cycle 0.00, fan-in 0.00, fan-out 0.00, gather-scatter 0.00, random 0.00, scatter-gather 0.00, stack 0.00						

validity exposed a defect: counterfactuals aimed at the raw score of the calibrator's breakpoint, but the calibrator interpolates between breakpoints, so the proposed change left the calibrated score above the threshold. As shipped, 0 of the 103 counterfactuals flipped the decision; aimed at the interpolated crossing, 103 of 103 do. The correction was registered as a protocol amendment before the experiment ran. Any system that derives counterfactual targets from an isotonic calibrator can make the same mistake.

Coverage is the larger limitation: the other 69% of alerts have no single-feature counterfactual over the actionable features, because typical transfer size dominates their score. Under 5% noise in non-actionable features, after Artelt et al. [34], the contribution vector moves by a median of 0.026 (relative L_1) and the top three features never change for the median alert; for a linear model this is close to guaranteed, so it mainly checks the implementation. A scorer retrained on disjoint seeds, after Movin et al. [35], ranks the contributions similarly (median Spearman 0.92) and still raises 87% of the alerts, but it proposes the same first counterfactual for only 29% of them, so a counterfactual describes this model, not the ring. Against simulator ground truth, the members shown in the evidence subgraph of a correct alert are the true ring's (median precision 1.00, coverage 1.00). Because that subgraph is built from the flagged ring's members, this checks that the evidence shows the right accounts; it does not test whether the evidence explains the score. A deletion test would: remove the shared-attribute links or the internal transfers that the evidence lists, recompute the ring's features and score, and compare the drop with removing as many links of the same members that the evidence does not list. We have not run it.

D. RQ9: a learned baseline and evasion strength

To test whether the rule stage is needed, we trained gradient boosting on the same 19 ring features over rule-free candidate groups (Louvain communities without a density filter). Training, calibration and threshold selection used the shipped scorer's protocol. On the test streams of RQ1 it reaches F1 0.555 against 0.731 (paired Wilcoxon $p < 0.001$, rank-biserial -0.85). Precision is not the problem: it is 0.94 against 0.76, with a third of the alerts. Recall is: 0.38 against 0.74. The gap is largest where hubs and evasion matter (`hub_heavy`: 0.40 against 0.79; `adversarial`: 0.24 against 0.59).

Turning up camouflage strength, after Liu et al. [4] and Hu et al. [8], separates the methods further. With up to eight decoy edges per ring action, the proposed method's recall of the decoy rings stays between 0.74 and 0.82, while Louvain falls to 0.00 and the learned baseline to 0.05. Slow rings remain the weakness found in RQ3: the proposed method recalls at most 0.25 of them, and static Cypher with a degree cap of ten recovers more (0.42–0.75), though at a precision near 0.04. In the 60 s scenario, strong slow and identity-rotation settings produce few rings, so those two sweeps have little statistical power. We report them as indicative.

VII. THREATS TO VALIDITY

Simulator realism. All main results come from our simulator. Hub sizes, ring behaviour and evasion strategies follow our configuration, not observed fraud. A detector tuned to this generator may do worse on real traffic, and the absolute numbers should not be read as field performance.

Synthetic labels. Ground truth is the set of rings the simulator planted. Benign groups that happen to look like rings count as false positives, and planted rings that never become visible in the stream still count against recall; both directions affect every method equally.

Public data. The public datasets we could obtain carry no device, card or IP links, so they cannot test hub-aware weighting at all; on them only the transaction detectors run (Section V-F). Only AMLworld has group labels, and we could replay only a four-day slice with a one-hour window. Elliptic and the AMLSim samples turned out to be outside what the transaction rules can detect, for structural reasons we describe. A real-world test of hub-aware weighting would need data with shared-attribute links, which we did not find in public form with group labels. The IEEE-CIS fraud data (https://ieee-dataport.org/documents/ieee-cis-fraud-detection) comes closest: e-commerce transactions with card, address and e-mail-domain fields and masked identity and device fields, labelled per transaction. It would let the shared-attribute detectors and the hub weight run on real links and measure false alarms around busy attributes, but without ring labels it cannot measure ring recall.

Tuning. Thresholds were selected on validation seeds of the same generator; the test seeds are independent draws but from the same distribution. In the weight-function comparison, three of the five selected edge thresholds lie on the edge of

the grid, so those functions might do somewhat better with a wider grid. The scorer was trained on the same generator and scenarios it is tested on; leaving one scenario out costs little (Section III), but a scenario type the generator cannot produce is not covered by that test.

Scale. The largest graph we measured has 100,000 users. A million users was not feasible on the 7.7 GB test machine: at 100,000 users less than 0.3 GB of memory was free and one full recomputation already took over 20 s per batch. In the scale scenario, attribute counts grow with the number of users while hub sizes stay fixed; graphs whose hubs grow with the population would give larger dirty regions. The 250-user first-hop limit was tested with hubs of up to 10,000 accounts, five seeds per size, on one scenario; its harmlessness rests on the hub-aware weight keeping a single busy attribute below the edge threshold, and would not hold for a weight or threshold that does not.

Latency. The latencies of Section V come from the in-memory store on a laptop. Section VI repeats the measurement on Neo4j and Redis Streams. The detected rings are identical, but detection is several times slower and throughput is bounded by the store. Both figures describe one laptop running Docker, not a server.

Baselines. The scored baselines of Table II use the shipped scorer, which was trained on candidates of the proposed pipeline; a scorer trained on each baseline's own candidates might do better for it. The learned baseline of Section VI is a tabular model over Louvain groups, so its weakness under decoy edges is partly the grouping's. We did not run a graph neural network. A GraphSAGE node classifier with a fixed rule for turning node scores into rings, as used by Badjie et al. [25] and Kumar et al. [26], is the obvious next comparison, together with node-level AUC on public benchmarks. Our node-level numbers are not comparable to those benchmarks.

Calibration. The shipped scorer is miscalibrated at the ring level (Section VI). Its scores rank alerts well but should not be read as probabilities until a ring-level calibrator replaces it. We did not re-run the main tables with the ring-level refit: it is a monotone map of the same scores, so with the alert threshold selected on the validation seeds it offers the same set of ranked operating points and changes only which one the grid picks, not the ranking behind Tables II and V.

Attacker model. The adaptive attacker only shifts its mix among the implemented evasion strategies. We did not test a white-box attacker that exploits the detector's parameters: rings smaller than 3 or larger than 25 members, sharing patterns whose edge density stays below 0.75, transfers kept under the cap $\beta_{\max}$ of the transaction term, or one distinct hub per pair of members. Each of these would bypass a fixed rule of our detector, so the evasion results are a lower bound on what an informed attacker achieves.

Streaming load. "Streaming" here means micro-batches on a paced simulated stream. Accuracy falls as the event rate rises (F1 0.61 at 10 events/s, 0.43 at 100), and on Neo4j the store saturates at 35–52 events per second on the test laptop. Cheap engineering steps we did not test are reading each region with one batched query instead of several round trips, and caching attribute degrees between batches.

VIII. Conclusion

Keeping links through busy shared attributes, instead of capping them, lets a streaming detector reach a recall on rings that share busy infrastructure that no cap reaches. The shape of the discount matters little: a constant weight with its own threshold did as well as $1/\ln(1+d)$, while strong discounts and soft caps lost the rings hidden behind hubs. On its own the weighting is not precise, and a calibrated scorer over ring features supplies most of the precision. Given the same scorer, the degree-capped rules reach F1 0.56 against 0.73 for the full pipeline, so the scorer accounts for most of the gain over the original rules, and at the best operating point on shared-attribute rings the weight and the cap are not distinguishable on F1. Restricting detection to the region a micro-batch touches gave the same rings as full recomputation in every accuracy run. On small graphs it is no cheaper than recomputing on a 1 s timer, and it detects no faster; its value is at scale, where at 20,000 users with growing hubs it costs 173 ms per simulated second against 4,170 ms for a 5 s timer, which can no longer keep up. Rings that rotate identities or act slowly evade all methods we tested, and all of our positive results are on synthetic data. On Neo4j and Redis Streams the detector flags exactly the same rings, at the cost of slower detection and a throughput bounded by the store. A learned model over rule-free groups is more precise but finds half as many rings, and it collapses under decoy edges, which the hub-aware rules withstand. Measuring the explanations found and fixed a calibration defect that had made every counterfactual invalid; after the fix every counterfactual flips the decision, but only 81 of 262 alerts have one, and a retrained model usually proposes a different one. The evidence shows the flagged accounts; whether it explains the score is untested. The next steps are data with real shared-attribute links, a ring-level calibrator, a GraphSAGE baseline, multi-feature counterfactuals, a deletion test of the evidence, and longer runs for slow and identity-rotating rings.

Code, scenarios, models and the scripts that regenerate every table and figure are in the project repository (`REPRODUCE.md`).

References

[1] T. Pourhabibi, K.-L. Ong, B. H. Kam, and Y. L. Boo, "Fraud detection: A systematic literature review of graph-based anomaly detection approaches," *Decision Support Systems*, vol. 133, p. 113303, 2020.

[2] P. Pan, "GB-FRD: Graph-based fraud ring detection in P2P payment networks," in *6th International Conference on Computer Vision and Data Mining (ICCVDM)*, 2025, pp. 457–462.

[3] H. Yin, Z. Zhang, Z. Wang, Y. Özyurt, W. Liang, W. Dong, Y. Zhao, and Y. Shan, "Behavioral graph fraud detection in e-commerce," in *IEEE International Conference on Data Mining Workshops (ICDMW)*, 2022, pp. 769–776.

[4] Z. Liu, J. Gao, H. Yu, and X. Luo, "A robust graph fraud detection model based on adversarial reweighting," *IEEE Transactions on Computational Social Systems*, vol. 12, no. 6, pp. 5213–5224, 2025.

[5] B. Zhang, H. Liu, and Y. Zhang, "Research on an incremental community detection algorithm for large-scale dynamic complex network," in *IEEE 3rd International Conference on Big Data Science and Engineering (ICBDSE)*, 2026.

[6] T. Abughofa, A. A. Harby, H. Isah, and F. Zulkernine, "Incremental community detection in distributed dynamic graph," in *IEEE 7th International Conference on Big Data Computing Service and Applications (BigDataService)*, 2021, pp. 50–59.

[7] B. Hooi, H. A. Song, A. Beutel, N. Shah, K. Shin, and C. Faloutsos, "FRAUDAR: Bounding graph fraud in the face of camouflage," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2016, pp. 895–904.

[8] P. Hu, Y. Qin, J. Wu, and Z. Yang, "HHGFD: Graph fraud detection through heterogeneity and heterophily characterization," in *IEEE 37th International Conference on Tools with Artificial Intelligence (ICTAI)*, 2025, pp. 818–825.

[9] J. Lv, Q. Du, Y. Zhang, Y. Han, and S. Li, "Multi-scale adaptive neighborhood awareness transformer for graph fraud detection," in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2026, pp. 6076–6080.

[10] X. Li, X. Jiang, Z. Wang, C. Liu, Z. Ni, H. Wan, and X. Zhao, "Adaptive Gaussian mixture model with hierarchical propagation for one-class graph fraud detection," in *IEEE International Conference on Multimedia and Expo (ICME)*, 2025.

[11] B. Giles, B. Jeudy, C. Largeron, and D. Saboul, "Suspicious: A resilient semi-supervised framework for graph fraud detection," in *IEEE 35th International Conference on Tools with Artificial Intelligence (ICTAI)*, 2023, pp. 212–220.

[12] H. Peng, J. Zhao, L. Li, Y. Ren, and S. Zhao, "One-class adversarial fraud detection nets with class specific representations," *IEEE Transactions on Network Science and Engineering*, vol. 10, no. 6, pp. 3793–3803, 2023.

[13] H. Yu, W. Liu, N. Zhu, P. Li, and X. Luo, "IN-GFD: An interpretable graph fraud detection model for spam reviews," *IEEE Transactions on Artificial Intelligence*, vol. 5, no. 10, pp. 5325–5339, 2024.

[14] M. Charikar, "Greedy approximation algorithms for finding dense components in a graph," in *Approximation Algorithms for Combinatorial Optimization (APPROX)*, ser. Lecture Notes in Computer Science, 2000, pp. 84–95.

[15] W. Feng, S. Liu, D. Koutra, and X. Cheng, "Unified dense subgraph detection: Fast spectral theory based algorithms," *IEEE Transactions on Knowledge and Data Engineering*, vol. 36, no. 3, pp. 1356–1370, 2024.

[16] A. Karaaslanli, P. A. Traganitis, and A. Konar, "Identifying adversarial attacks in crowdsourcing via dense subgraph detection," in *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, 2025.

[17] J. Jiang, Y. Li, B. He, B. Hooi, J. Chen, and J. K. Z. Kang, "Spade: A real-time fraud detection framework on evolving graphs," *Proceedings of the VLDB Endowment*, vol. 16, no. 3, pp. 461–469, 2022.

[18] A. Ahmad, L. Yuan, D. Yan, G. Guo, J. Chen, and C. Zhang, "Accelerating k-core decomposition by a GPU," in *IEEE 39th International Conference on Data Engineering (ICDE)*, 2023, pp. 1818–1831.

[19] L. Li, H. Chen, P. Li, J. Han, G. Wang, and G. Zhang, "The k-core decomposition algorithm under the framework of GraphBLAS," in *IEEE High Performance Extreme Computing Conference (HPEC)*, 2021.

[20] L. Dhulipala, Q. C. Liu, S. Raskhodnikova, J. Shi, J. Shun, and S. Yu, "Differential privacy from locally adjustable graph algorithms: k-core decomposition, low out-degree ordering, and densest subgraphs," in *IEEE 63rd Annual Symposium on Foundations of Computer Science (FOCS)*, 2022, pp. 754–765.

[21] V. D. Blondel, J.-L. Guillaume, R. Lambiotte, and E. Lefebvre, "Fast unfolding of communities in large networks," *Journal of Statistical Mechanics: Theory and Experiment*, vol. 2008, no. 10, p. P10008, 2008.

[22] X. Li, S. Dong, and S. Li, "A differentially private non-overlapping community detection method based on improved Louvain algorithm," in *6th International Conference on Data Science and Information Technology (DSIT)*, 2023, pp. 7–12.

[23] L. A. Adamic and E. Adar, "Friends and neighbors on the Web," *Social Networks*, vol. 25, no. 3, pp. 211–230, 2003.

[24] S. Varma, S. Shivam, A. Thumu, A. Bhushanam, and D. Sarkar, "Jaccard based similarity index in graphs: A multi-hop approach," in *IEEE Delhi Section Conference (DELCON)*, 2022.

[25] S. Badjie, A. A. Susom, B. D. Sarker, N. bin Sharif, and S. A. Ashraf, "Ensembles or graph neural networks? Detecting illicit Bitcoin transactions under temporal drift," in *6th International Conference on Soft Computing for Security Applications (ICSCSA)*, 2026, pp. 557–561.

[26] R. N. Kumar and N. Oláh, "Temporal-aware graph attention network with XGBoost ensemble for blockchain transaction fraud detection," in *IEEE 4th Conference on Information Technology and Data Science (CITDS)*, 2026, pp. 331–334.

[27] B. Lu, T. Xiao, Y. Zhao, and L. Su, "LAGCN-AD: An LSH-attention-based graph convolutional network for blockchain anomaly detection," in *IEEE 6th International Conference on Electronic Technology, Communication and Information (ICETCI)*, 2026, pp. 131–135.

[28] S. B. Shah, "Advanced machine learning models for anti-money laundering (AML): Improving detection accuracy and efficiency," in *IEEE 1st International Conference on Secure IoT, Assured and Trusted Computing (SATC)*, 2025.

[29] E. Altman, J. Blanuša, L. von Niederhäusern, B. Egressy, A. Anghel, and K. Atasu, "Realistic synthetic financial transactions for anti-money laundering models," in *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, 2023.

[30] B. Zadrozny and C. Elkan, "Transforming classifier scores into accurate multiclass probability estimates," in *Proceedings of the 8th ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 2002, pp. 694–699.

[31] A. Niculescu-Mizil and R. Caruana, "Predicting good probabilities with supervised learning," in *Proceedings of the 22nd International Conference on Machine Learning (ICML)*, 2005, pp. 625–632.

[32] T. Azbergen, A. Tleubayeva, and A. Aituov, "Calibration-aware gradient boosting with temporal behavioral features for anti-money laundering detection," in *IEEE 6th International Conference on Smart Information Systems and Technologies (SIST)*, 2026.

[33] Z. Ding, X. Han, P. Liu, and M. Niethammer, "Local temperature scaling for probability calibration," in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2021, pp. 6869–6879.

[34] A. Artelt, V. Vaquet, R. Velioglu, F. Hinder, J. Brinkrolf, M. Schilling, and B. Hammer, "Evaluating robustness of counterfactual explanations," in *IEEE Symposium Series on Computational Intelligence (SSCI)*, 2021.

[35] M. Movin, F. Siciliano, R. Ferreira, F. Silvestri, and G. Tolomei, "Consistent counterfactual explanations via anomaly control and data coherence," *IEEE Transactions on Artificial Intelligence*, vol. 6, no. 4, pp. 794–804, 2025.

[36] B. Meulemeester, R. M. B. de Oliveira, Y. He, and D. Martens, "Calculating and visualizing feature importance values for counterfactual explanations," *IEEE Access*, vol. 14, pp. 50 691–50 703, 2026.

[37] K. Bora, E. Kartsakli, and E. Quinõnes Moreno, "HYADES: Adaptive consumer tuning in Redis Streams via hybrid expert selection," in *IEEE 19th International Conference on Cloud Computing (CLOUD)*, 2026, pp. 518–528.